

The background of the entire page is a close-up, warm-toned photograph of several hands of different skin tones stacked on top of each other, palms facing down. This visual metaphor represents teamwork, unity, and shared effort.

INTEGRA
SENAI

REGRAFIK

SENAI Serviço Nacional
de Aprendizagem
Industrial

FICHA CATALOGRÁFICA

Título: ReGraphik — Sistema de Gestão de Estoque Reverso

Autores: Bruno Maia, Otávio henrique, Lucas Aquino, Luna Beatriz, Kaio Alves

Curso: Técnico em Desenvolvimento de Sistemas — SENAI

Ano: 2025

Repositório: <https://github.com/BrunoMaiaSenai/ReGraphikApp>

SUMÁRIO

RESUMO 4

ABSTRACT 5

1 INTRODUÇÃO 6

1.1 Justificativa.....	6
1.2 Objetivos.....	6
1.3 Metodologia.....	7

2 MINI MUNDO DA DEMANDA 7

2.1 Contexto Organizacional.....	7
2.2 Atores do Sistema.....	8
2.3 Regras de Negócio.....	9

3 MODELAGEM DO BANCO DE DADOS 10

3.1 Modelo Conceitual (DER).....	10
3.2 Modelo Lógico.....	13

4 ARQUITETURA DO SISTEMA 15

4.1 Visão Geral da Arquitetura.....	15
4.2 Padrão MVVM no Cliente WPF.....	15
4.3 Estrutura do Repositório.....	16
4.4 Diagrama de Integração.....	17

5 API REST — ENDPOINTS E INTEGRAÇÃO 18

5.1 Endpoints Implementados.....	18
5.2 Fluxo Detalhado — Busca de Pontos de Coleta.....	19
5.3 Documentação Swagger.....	20
5.4 Configuração CORS e Segurança.....	21

6 VIABILIDADE TÉCNICA 22

7 ARQUITETURA DO PROJETO 26

8 MÓDULOS DO SISTEMA 27

9 PROCESSO DE DESENVOLVIMENTO 29

10 MODEL CANVAS 30

11 VIABILIDADE ECONÔMICA 30

11.1 Custos de Desenvolvimento.....	31
11.2 Custos Operacionais.....	32
11.3 Benefícios e Retorno Esperado.....	33
11.4 Análise Custo-Benefício.....	34

12 ARTEFATOS TÉCNICOS ENTREGUES 34

13 REQUISITOS 35

14 CASO DE USO 37

15 HISTORICO DE EVOLUÇÃO E ATUALIZAÇÃO DO PROJETO 38**16 CONSIDERAÇÕES FINAIS 39****REFERÊNCIAS 39****RESUMO**

O presente trabalho descreve o desenvolvimento do ReGraphik, um sistema de software voltado à gestão de estoque reverso no setor gráfico. O projeto surge como resposta ao descarte inadequado de resíduos sólidos — papel, cartão, vinil, lona e PVC — gerados rotineiramente por empresas do segmento gráfico, em especial pela empresa AML. A ausência de um processo estruturado para catalogar e reaproveitar esses materiais representa desperdício econômico e impacto ambiental desnecessário.

A solução proposta é composta por dois subsistemas integrados: uma API REST desenvolvida em ASP.NET Core, responsável pelo processamento das regras de negócio e pela persistência dos dados no Firebase Realtime Database; e um cliente desktop construído em WPF (.NET), estruturado segundo o padrão arquitetural MVVM. A integração com a Google Maps Places API permite a localização georreferenciada de pontos de coleta seletiva próximos ao usuário.

O sistema contempla funcionalidades de cadastro e rastreamento de resíduos, motor de sugestões de reaproveitamento por tipo de material, dashboard de indicadores de desempenho e geração de relatórios operacionais. A modelagem relacional é composta por cinco entidades principais: CadastroUsuarios, TipoMaterial, CadastroResiduos, Sugestoes e SugestoesResiduos.

O desenvolvimento utilizou boas práticas de engenharia de software, incluindo injeção de dependências, separação de responsabilidades em camadas, programação assíncrona (async/await) e documentação da API via Swagger/OpenAPI. O repositório acumula mais de 214 commits até o momento, evidenciando histórico de desenvolvimento consistente e colaborativo.

ABSTRACT

This paper describes the development of ReGraphik, a software system aimed at reverse stock management in the graphic arts sector. The project arises as a response to the improper disposal of solid waste — paper, cardboard, vinyl, canvas, and PVC — routinely generated by companies in the graphic segment, particularly by the company AML. The absence of a structured process to catalog and reuse these materials represents economic waste and unnecessary environmental impact.

The proposed solution consists of two integrated subsystems: a REST API developed in ASP.NET Core, responsible for processing business rules and persisting data in Firebase Realtime Database; and a desktop client built in WPF (.NET), structured according to the MVVM architectural pattern. Integration with the Google Maps Places API enables georeferenced location of selective collection points near the user.

The system includes features for waste registration and tracking, a reuse suggestion engine by material type, a performance indicator dashboard, and operational report generation. The relational model consists of five main entities: CadastroUsuarios, TipoMaterial, CadastroResiduos, Sugestoes, and SugestoesResiduos.

1 INTRODUÇÃO

O setor gráfico é um dos segmentos industriais que mais gera resíduos sólidos no Brasil. Empresas de médio e grande porte produzem diariamente toneladas de aparas de papel, retalhos de vinil, restos de cartão, lona e PVC que, na ausência de processos estruturados de gestão, são simplesmente descartados. Esse comportamento acarreta custos financeiros com o descarte, impacto ambiental mensurável e, sobretudo, desperdício de matéria-prima que poderia ser reintegrada à cadeia produtiva.

O presente trabalho apresenta o ReGraphik, sistema de software desenvolvido como Trabalho de Conclusão de Curso do Curso Técnico em Desenvolvimento de Sistemas do SENAI, com o objetivo de propor uma solução tecnológica ao problema descrito acima. O sistema oferece um ciclo completo de gestão de estoque reverso: desde o cadastro e classificação dos resíduos gerados, passando pela sugestão de rotas de reaproveitamento, até a localização de pontos de coleta na região.

1.1 JUSTIFICATIVA

A ausência de ferramentas digitais específicas para o controle de resíduos no setor gráfico obriga as empresas a recorrerem a planilhas manuais ou a simplesmente ignorar o potencial reaproveitável de seus descartes. O projeto ReGraphik nasce da necessidade real identificada na empresa AML, onde o gerenciamento inadequado de resíduos representa perda econômica estimada e impacto ambiental evitável. A digitalização deste processo, combinada com sugestões automatizadas de reaproveitamento e integração com serviços de geolocalização, representa um avanço concreto em direção à economia circular no segmento gráfico.

1.2 OBJETIVOS

1.2.1 OBJETIVO GERAL

Desenvolver um sistema de gestão de estoque reverso para o setor gráfico, composto por uma API REST em ASP.NET Core e um cliente desktop WPF, estruturado no padrão MVVM, com integração a serviços externos de banco de dados em nuvem e geolocalização.

1.2.2 OBJETIVOS ESPECÍFICOS

- Modelar e implementar o banco de dados relacional com cinco entidades (CadastroUsuarios, TipoMaterial, CadastroResiduos, Sugestoes e SugestoesResiduos);
- Desenvolver uma API REST documentada (Swagger) com endpoints CRUD para todas as entidades do domínio;
- Implementar o padrão MVVM no cliente WPF, com BaseViewModel, RelayCommand e bindings declarativos;
- Integrar a Google Maps Places API para localização de pontos de coleta georreferenciados;
- Persistir os dados no Firebase Realtime Database, garantindo sincronização em nuvem;
- Documentar o sistema conforme os artefatos técnicos exigidos: Mini Mundo, Modelo Conceitual, Modelo Lógico e Modelo Físico.

1.3 METODOLOGIA

O projeto foi desenvolvido de forma iterativa e incremental, com divisão em três fases: configuração do ambiente e modelagem inicial (Fase 1), desenvolvimento da interface e integração com mapas (Fase 2) e construção da API REST e integração completa (Fase 3). O controle de versão foi realizado via Git/GitHub, com 182 commits registrados até a entrega desta documentação.

2 MINI MUNDO DA DEMANDA

Este capítulo descreve o contexto organizacional que motivou o desenvolvimento do ReGraphik, os atores envolvidos e as regras de negócio que orientaram a modelagem do sistema.

2.1 CONTEXTO ORGANIZACIONAL

Para sanar os gargalos da empresa AML, o fluxo operacional do Mini Mundo do sistema ReGraphik foi padronizado sob uma arquitetura de microsserviços e tabelas normalizadas. O ciclo de vida inicia-se com o mapeamento e validação relacional das entidades. O Administrador do sistema pré-configura a tabela de TipoMaterial. Quando o operador executa o descarte fabril, o sistema captura suas credenciais a partir da

tabela CadastroUsuarios e cria uma instância na tabela CadastroResiduos, associando-a obrigatoriamente às chaves estrangeiras (FK) correspondentes. O motor de busca realiza o cruzamento de cardinalidade 1:N com as tabelas de Sugestoes por meio da entidade associativa N:N SugestoesResiduos, registrando a data exata da aplicação da economia circular. Caso o material seja classificado como sem reuso interno, a API intercepta a localização geográfica, busca pontos externos via mapas, valida a redundância dos dados existentes e atualiza o inventário geral em tempo real na nuvem Firebase.

2.2 ATORES DO SISTEMA

2.2.1 EMPRESA (AML OU OUTRAS GRÁFICAS)

- Cadastra usuários no sistema;
- Utiliza o sistema como ferramenta de gestão operacional.

2.2.2 USUÁRIO DO SISTEMA

- Realiza login com credenciais cadastradas;
- Cadastra resíduos gerados no processo produtivo;
- Consulta o estoque reverso atualizado;
- Aplica sugestões de reaproveitamento aos resíduos cadastrados;
- Gera relatórios operacionais.

2.2.3 ADMINISTRADOR

- Gerencia usuários do sistema;
- Gerencia tipos de materiais cadastrados;
- Importa sugestões em lote via planilha.

2.2.4 SISTEMA REGRAPHIK

- Controla o estoque reverso de resíduos;
- Sugere formas de reaproveitamento por tipo de material;
- Gera dashboards e relatórios gerenciais.

2.3 REGRAS DE NEGÓCIO

1. Um usuário pertence obrigatoriamente a uma empresa;
2. Um resíduo deve ser classificado por um tipo de material;
3. Um resíduo pode receber várias sugestões de reaproveitamento;
4. Uma sugestão pode ser aplicada a múltiplos resíduos (relação N:N);
5. O sistema registra automaticamente a data de cadastro de cada entidade;
6. O sistema controla o status do resíduo com os seguintes estados: Em Estoque, Reaproveitado e Descartado;
7. A exclusão de resíduos é restrita ao perfil Administrador e gera auditoria.

3 MODELAGEM DO BANCO DE DADOS

Este capítulo apresenta os três níveis de modelagem do banco de dados do sistema ReGraphik: conceitual, lógico e físico, conforme as metodologias de modelagem relacional adotadas no curso.

3.1 MODELO CONCEITUAL (DER)

O modelo conceitual representa as entidades do domínio e seus relacionamentos em nível de abstração, sem preocupação com tipos de dados ou chaves de implementação. A modelagem segue a notação do BRModelo, utilizando Diagrama Entidade-Relacionamento (DER).

3.1.1 ENTIDADES E ATRIBUTOS

Atributo	Tipo / Papel	Observação
id_usuario	PK	Identificador único do usuário
nome	Atributo	Nome completo do usuário
cpf	Atributo	CPF do usuário (único)
email	Atributo	E-mail de contato
login	Atributo	Nome de usuário para acesso
senha	Atributo	Senha criptografada
dataCadastro	Atributo	Data/hora de criação do registro

Atributo	Tipo / Papel	Observação
id_tipoMaterial	PK	Identificador único do tipo de material
descricaoMaterial	Atributo	Descrição do tipo (papel, vinil, cartão, etc.)

Atributo	Tipo / Papel	Observação
id_residuo	PK	Identificador único do resíduo
id_usuario	FK	Referência ao usuário cadastrante
id_tipoMaterial	FK	Referência ao tipo de material
origem	Atributo	Origem do resíduo (projeto/setor)
especificacao	Atributo	Especificação técnica do material
projeto	Atributo	Projeto de origem do resíduo
quantidade	Atributo	Quantidade disponível em estoque
dataCadastro	Atributo	Data/hora do cadastro
condicao	Atributo	Estado físico do material
dimensoesCm	Atributo	Dimensões em centímetros
dimensoesLm	Atributo	Dimensões em metros lineares
observacao	Atributo	Observações adicionais
anexo	Atributo	Caminho do arquivo anexado
status	Atributo	Em Estoque Reaproveitado Descartado

Atributo	Tipo / Papel	Observação
id_sugestao	PK	Identificador único da sugestão
id_tipoMaterial	FK	Tipo de material ao qual a sugestão se aplica

Atributo	Tipo / Papel	Observação
descricaoSugestao	Atributo	Descrição da sugestão de reaproveitamento

Atributo	Tipo / Papel	Observação
id	PK	Identificador único do relacionamento
idCadastroResiduo	FK	Referência ao resíduo
idSugestao	FK	Referência à sugestão
dataAplicacao	Atributo	Data em que a sugestão foi aplicada ao resíduo

3.1.2 RELACIONAMENTOS

Os relacionamentos entre as entidades do sistema são definidos a seguir:

Relacionamento	Cardinalidade	Semântica
USUARIO — CADASTRO_RESIDUOS	1 : N	Um usuário pode cadastrar vários resíduos.
TIPO_MATERIAL — CADASTRO_RESIDUOS	1 : N	Um tipo de material classifica vários resíduos.
TIPO_MATERIAL — SUGESTOES	1 : N	Um tipo de material pode ter várias sugestões.
CADASTRO_RESIDUOS — SUGESTOES	N : N	Resolvido por SUGESTOES_RESIDUOS: um resíduo recebe várias sugestões, e uma sugestão pode ser aplicada a vários resíduos.

3.2 MODELO LÓGICO

O modelo lógico transforma as entidades conceituais em tabelas relacionais, definindo chaves primárias (PK), chaves estrangeiras (FK) e os tipos de dados de cada atributo.

Tipagem descrita como INT / String foi unificada nas chaves primárias e estrangeiras (ID, IdUsuario, IdTipoMaterial) para aceitar tanto inteiros, impedindo quebras de integridade referencial em chamadas assíncronas

3.2.1 TABELA: CADASTROUSUARIOS

Coluna	Chave / Relacionamento	Tipo	Descrição
ID	PK	INT / String	Identificador único
Nome	—	VARCHAR(100)	Nome completo
CPF	—	VARCHAR(14)	CPF único do usuário
Email	—	VARCHAR(150)	E-mail
Login	—	VARCHAR(50)	Login de acesso (único)
Senha	—	VARCHAR(255)	Hash da senha
DataCadastro	—	DATETIME	Data/hora de cadastro

3.2.2 TABELA: TIPOMATERIAL

Coluna	Chave / Relacionamento	Tipo	Descrição
Id	PK	INT / String	Identificador único
DescricaoMaterial	—	VARCHAR(100)	Descrição do material

3.2.3 TABELA: CADASTRORESIDUOS

Coluna	Chave / Relacionamento	Tipo	Descrição
ID	PK	INT / String	Identificador único
IdUsuario	FK → CadastroUsuarios(ID)	INT / String	Usuário cadastrante
IdTipoMaterial	FK → TipoMaterial(Id)	INT / String	Tipo do material
Origem	—	VARCHAR(100)	Origem do resíduo
Especificacao	—	TEXT	Especificação técnica
Projeto	—	VARCHAR(100)	Projeto de origem
Quantidade	—	DECIMAL	Quantidade em estoque
DataCadastro	—	DATETIME	Data/hora de cadastro
Condicao	—	VARCHAR(50)	Estado físico do material
DimensoesCm	—	VARCHAR(50)	Dimensões (cm)
DimensoesLm	—	VARCHAR(50)	Dimensões (metros lineares)
Observacao	—	TEXT	Observações livres
Anexo	—	VARCHAR(255)	Caminho do arquivo
Status	—	ENUM	Em Estoque Reaproveitado Descartado

3.2.4 TABELA: SUGESTOES

Coluna	Chave / Relacionamento	Tipo	Descrição
Id	PK	INT / String	Identificador único

Coluna	Chave / Relacionamento	Tipo	Descrição
IdTipoMaterial	FK → TipoMaterial(Id)	INT / String	Tipo de material relacionado
DescricaoSugestao	—	TEXT	Descrição da sugestão de reaproveitamento

3.2.5 TABELA: SUGESTOESRESIDUOS

Coluna	Chave / Relacionamento	Tipo	Descrição
ID	PK	INT / String	Identificador único
IdCadastroResiduo	FK CadastroResiduos(ID) →	INT / String	Resíduo vinculado
IdSugestao	FK → Sugestoes(Id)	INT / String	Sugestão aplicada
DataAplicacao	—	DATETIME	Data em que a sugestão foi aplicada

4 ARQUITETURA DO SISTEMA

O ReGraphik adota uma arquitetura em camadas, separando claramente as responsabilidades entre o cliente desktop, a API REST e o banco de dados em nuvem. Esta seção descreve cada componente e seus padrões de design.

4.1 VISÃO GERAL DA ARQUITETURA

O sistema é composto por três camadas principais:

- Cliente WPF (Frontend Desktop) — interface do usuário, construída em XAML/C# com padrão MVVM;
- API REST em ASP.NET Core (Backend) — processamento das regras de negócio e exposição de endpoints HTTP;
- Firebase Realtime Database (Persistência) — banco de dados NoSQL em nuvem, mantido pelo Google.

Adicionalmente, o sistema integra a Google Maps Places API, consumida tanto pela API REST (para busca e persistência de pontos de coleta) quanto pelo cliente desktop (para renderização do mapa via Leaflet.js em um componente WebBrowser/WebView2).

4.2 PADRÃO MVVM NO CLIENTE WPF

O cliente desktop segue rigorosamente o padrão MVVM (Model-View-ViewModel), que estabelece três camadas de responsabilidade distintas:

Camada	Componentes	Responsabilidade
VIEW	MainWindow, DashboardPage, ResiduosPage, EstoqueReversoPage, MapaPage, RelatoriosPage (XAML)	Apresentação visual; bindings declarativos; zero lógica de negócio; zero code-behind.
VIEWMODEL	BaseViewModel, ResiduoViewModel, LoginViewModel, MapViewModel (C#)	Lógica de apresentação; implementa INotifyPropertyChanged; expõe ObservableCollection e RelayCommand para a View.
MODEL	Usuario, Residuo, Sugestao, SugestaoResiduo, PontosColeta (POCOs C#)	Entidades do domínio puras, sem dependência de UI ou infraestrutura.
SERVICE LAYER	GooglePlacesService, ApiService (C#)	Lógica HTTP, integração com API REST e Google Maps; injetado nos ViewModels via construtor.

4.2.1 BASEVIEWMODEL E INOTIFYPROPERTYCHANGED

Todos os ViewModels herdam de BaseViewModel, que implementa INotifyPropertyChanged. Isso permite que a View reaja automaticamente a mudanças de propriedades via Data Binding, sem acoplamento direto entre as camadas.

4.2.2 RELAYCOMMAND

O padrão Command é implementado pela classe RelayCommand, que encapsula delegates Action<object> e Func<object, bool> para execução e validação de comandos. Cada botão da interface é associado a um Command via binding XAML, eliminando o uso de event handlers no code-behind.

4.3 ESTRUTURA DO REPOSITÓRIO

A estrutura de diretórios do projeto é organizada da seguinte forma:

```
ReGraphikApp/
├── ApiRestReGraphik/           # Backend – ASP.NET Core REST API
│   ├── Controllers/           # Endpoints HTTP (CRUD completo)
│   ├── Services/              # Regras de negócio e acesso ao Firebase
│   ├── Models/                # Entidades do domínio
│   ├── Data/                  # Configuração do Firebase Client (DbReGraphik.cs)
│   └── Program.cs              # DI, Swagger, CORS
│
├── ReGraphik/                  # Frontend – WPF Desktop (MVVM)
│   ├── Views/Pages/           # Dashboard, Resíduos, Mapa, Relatórios
│   ├── ViewModels/            # BaseViewModel, ResiduoViewModel
│   ├── Models/                # Espelho das entidades do domínio
│   ├── Services/              # GooglePlacesService
│   └── Commands/              # RelayCommand
│
└── Modelagem/                 # PDFs: Conceitual, Lógico, Físico, MVVM
```

4.4 DIAGRAMA DE INTEGRAÇÃO

O fluxo de comunicação entre os componentes do sistema segue a sequência abaixo:

```
WPF CLIENT (View → ViewModel → Command)

|

| HTTP / REST (JSON)

↓

API REST — ApiRestReGraphik (ASP.NET Core)

Controllers → Services → Firebase SDK

|

| Firebase SDK (HTTPS)

↓

FIREBASE REALTIME DATABASE (NoSQL — Google Cloud)

|

| Google Maps Places API (HTTPS)

↓

GOOGLE MAPS PLACES API (NearbySearch / TextSearch)
```

5 API REST — ENDPOINTS E INTEGRAÇÃO

A API REST do ReGraphik é desenvolvida em ASP.NET Core e exposta no domínio <https://webregraphik.runasp.net>. Ela é responsável por receber as requisições do cliente WPF, aplicar as regras de negócio e persistir os dados no Firebase Realtime Database.

5.1 ENDPOINTS IMPLEMENTADOS

Método	Endpoint	Descrição
GET	/api/residuos	Lista todos os resíduos com filtros opcionais de status e tipo
GET	/api/residuos/{id}	Retorna resíduo específico com histórico de sugestões aplicadas

Método	Endpoint	Descrição
POST	/api/residuos	Cadastra novo resíduo — valida tipo, dimensões e usuário autenticado
PUT	/api/residuos/{id}/status	Atualiza status (ativo, em_uso, descartado) com log de alteração
DELETE	/api/residuos/{id}	Remove resíduo — requer perfil ADMIN, registra auditoria
GET	/api/usuarios	Lista usuários — restrito ao perfil ADMIN
POST	/api/sugestoes/aplicar	Aplica sugestão de reaproveitamento — gera registro SugestaoResiduo
GET	/api/coleta/proximos	Integra GooglePlacesService — retorna pontos por lat/Ing

5.2 FLUXO DETALHADO — BUSCA DE PONTOS DE COLETA

O módulo de mapa demonstra a integração completa entre as camadas do sistema. O fluxo ocorre em seis etapas:

8. O usuário clica em 'Buscar' na MapaPage (View XAML);
9. O evento aciona o BtnBuscar_Click no code-behind, que monta a URL da API com a cidade informada;
10. A API recebe a requisição e constrói a query para a Google Maps Places API;
11. O resultado retornado pelo Google Maps é persistido no Firebase (pontos_coleta);
12. A API retorna a lista de pontos (lat/Ing) como JSON para o cliente WPF;
13. O cliente WPF gera o HTML com Leaflet.js, grava em arquivo temporário e renderiza no componente WebBrowser.

5.3 DOCUMENTAÇÃO SWAGGER

A API é documentada via Swagger/OpenAPI (Swashbuckle). A interface Swagger UI permite explorar todos os endpoints, visualizar os modelos de requisição/resposta e

testar as chamadas diretamente no navegador. A documentação completa está disponível em /swagger após a inicialização da API.

A API expõe 5 controllers com operações CRUD completas.

A documentação interativa fica disponível via Swagger na raiz da aplicação ao rodar o projeto.

Usuário — [api/Usuario](#)

Método	Rota	Descrição
GET	/api/Usuario	Lista todos os usuários
GET	/api/Usuario/{id}	Obtém usuário por ID
POST	/api/Usuario	Cria novo usuário
PUT	/api/Usuario/{id}	Atualiza usuário existente
DELETE	/api/Usuario/{id}	Remove usuário

Pontos de Coleta — [api/PontosColeta](#)

Método	Rota	Descrição
GET	/api/PontosColeta	Lista todos os pontos cadastrados no Firebase
GET	/api/PontosColeta/{id}	Obtém ponto por ID
POST	/api/PontosColeta/google?cidade=...	Cadastra pontos via Google Maps Places API e salva no Firebase
POST	/api/PontosColeta	Cadastra novo ponto de coleta
PUT	/api/PontosColeta/{id}	Atualiza ponto existente

Método	Rota	Descrição
DELETE	/api/PontosColeta/{id}	Remove ponto de coleta

O endpoint `/google` valida se a cidade já está cadastrada no Firebase antes de consultar o Google Maps. Se autorizada, busca os pontos, salva automaticamente e retorna a lista com os IDs gerados.

Sugestão — `api/Sugestao`

Método	Rota	Descrição
GET	/api/Sugestao	Lista todas as sugestões
GET	/api/Sugestao/{id}	Obtém sugestão por ID
POST	/api/Sugestao	Cria nova sugestão
PUT	/api/Sugestao/{id}	Atualiza sugestão existente
DELETE	/api/Sugestao/{id}	Remove sugestão

Sugestão de Resíduos — `api/SugestaoResiduos`

Método	Rota	Descrição
GET	/api/SugestaoResiduos	Lista todas as sugestões aplicadas a resíduos
GET	/api/SugestaoResiduos/{id}	Obtém por ID
POST	/api/SugestaoResiduos	Registra aplicação de sugestão a um resíduo
PUT	/api/SugestaoResiduos/{id}	Atualiza registro
DELETE	/api/SugestaoResiduos/{id}	Remove registro

6 VIABILIDADE TÉCNICA

A análise de viabilidade técnica avalia se o sistema ReGraphik pode ser desenvolvido, implantado e mantido com os recursos tecnológicos disponíveis, considerando o contexto acadêmico do projeto e a realidade das empresas do setor gráfico a que se destina.

6.1 INFRAESTRUTURA E TECNOLOGIAS

Todas as tecnologias adotadas no ReGraphik são gratuitas, de ampla adoção no mercado e com documentação extensiva, eliminando barreiras de aprendizado e custo de licenciamento. A tabela a seguir resume a disponibilidade e maturidade de cada componente:

Componente	Licença	Maturidade	Observação
C# / .NET 8+	MIT (open-source)	Alta	Plataforma Microsoft com suporte LTS garantido
ASP.NET Core	MIT (open-source)	Alta	Framework web maduro, utilizado em produção globalmente
WPF	MIT (open-source)	Alta	Framework nativo Windows com base de código estável
Firebase Realtime DB	Gratuito (plano Spark)	Alta	Plataforma Google com 99,95% de SLA
Google Maps Places API	Gratuito até US\$200/mês	Alta	API estável com versão nova em produção
Leaflet.js	BSD-2-Clause	Alta	Biblioteca de mapas mais utilizada do mundo
Swagger/OpenAPI	Apache 2.0	Alta	Padrão de mercado para documentação de APIs REST

Git / GitHub	Gratuito (plano Free)	Alta	Plataforma de controle de versão líder de mercado
--------------	-----------------------	------	---

6.1.1 REQUISITOS MÍNIMOS DE HARDWARE

Para execução do sistema em ambiente de produção, os requisitos mínimos recomendados são:

- Processador: Intel Core i3 ou equivalente (2 GHz ou superior);
- Memória RAM: 4 GB (8 GB recomendado para ambiente de desenvolvimento);
- Armazenamento: 500 MB para instalação (dados persistidos em nuvem no Firebase);
- Sistema Operacional: Windows 10 / Windows 11 (64 bits) — obrigatório para o cliente WPF;
- Conexão com a internet: necessária para integração com Firebase e Google Maps Places API.

6.1.2 REQUISITOS DE SOFTWARE

- .NET 8 Runtime ou superior (redistribuível gratuito da Microsoft);
- Visual C++ Redistributable (dependência indireta do WPF);
- WebBrowser/WebView2 — componente nativo do Windows 10/11, necessário para renderização do mapa via Leaflet.js.

6.2 RISCOS TÉCNICOS E MITIGAÇÕES

Risco	Probabilidade	Impacto	Mitigação
Cota gratuita do Firebase excedida com alto volume de dados	Média	Alto	Migrar para plano Blaze (pay-as-you-go) ou adotar banco relacional próprio
Cota gratuita da Google Maps API excedida	Média	Médio	Implementar cache de resultados no Firebase e limitar chamadas por sessão

Dependência exclusiva de plataforma Windows para o cliente WPF	Baixa	Médio	Desenvolver versão web (Blazor) ou mobile (.NET MAUI) em iterações futuras
Ausência de autenticação JWT na API REST	Alta	Alto	Implementar JWT como prioridade na próxima sprint; já documentado no roadmap
Perda de conectividade com a internet	Baixa	Alto	Implementar cache local (SQLite) para funcionamento offline parcial
Quebra de contrato da Google Maps Places API	Muito Baixa	Médio	Interface IPlacesService abstrai o provedor, permitindo substituição sem impacto nas ViewModels

6.3 ESCALABILIDADE

A arquitetura adotada foi projetada para suportar crescimento gradual sem necessidade de refatoração estrutural. Os principais aspectos de escalabilidade são:

- A API REST em ASP.NET Core pode ser hospedada em múltiplas instâncias (horizontal scaling) com load balancer, sem alteração de código;
- O Firebase Realtime Database escala automaticamente conforme a demanda, sem configuração adicional;
- O padrão MVVM no cliente WPF permite adicionar novos módulos e funcionalidades sem modificar os existentes;
- A interface IPlacesService abstrai o provedor de mapas, permitindo substituição sem impacto nas ViewModels;
- A separação entre Controllers e Services na API facilita a extração de microserviços futuros.

Camada	Tecnologia	Versão / Detalhe
Linguagem	C# (.NET)	.NET 8 / .NET 9
Frontend Desktop	WPF — Windows Presentation Foundation	Windows-only

Camada	Tecnologia	Versão / Detalhe
Padrão de Projeto	MVVM (Model-View-ViewModel)	Padrão GoF
Backend / API	ASP.NET Core Web API	REST / JSON
Banco de Dados	Firebase Realtime Database	NoSQL — Google
Autenticação Firebase	Google Service Account (JSON)	OAuth 2.0
Mapas / Geolocalização	Google Maps Places API + Leaflet.js	WebBrowser/WebView2
Documentação da API	Swagger / OpenAPI	Swashbuckle
Controle de Versão	Git / GitHub	182+ commits
IDE de Desenvolvimento	Visual Studio 2022	Community Edition

6.1 C# E .NET

A linguagem C# oferece tipagem estática, suporte nativo a programação assíncrona (async/await) e o ecossistema .NET, que viabiliza tanto o desenvolvimento desktop (WPF) quanto o desenvolvimento de APIs web (ASP.NET Core) dentro de um mesmo ambiente de desenvolvimento. A escolha por uma única linguagem para frontend e backend reduz a curva de aprendizado e facilita o compartilhamento de entidades de domínio.

6.2 WPF E MVVM

O Windows Presentation Foundation é o framework nativo da Microsoft para desenvolvimento de aplicações desktop ricas em Windows. Sua compatibilidade com o padrão MVVM, por meio de Data Binding declarativo no XAML, torna-o ideal para projetos que exigem separação clara de responsabilidades entre interface e lógica.

6.3 ASP.NET CORE WEB API

O ASP.NET Core é um framework de alta performance para desenvolvimento de APIs HTTP multiplataforma. Sua arquitetura baseada em injeção de dependências facilita a modularização do código e a criação de testes unitários.

6.4 FIREBASE REALTIME DATABASE

O Firebase Realtime Database é um banco de dados NoSQL hospedado na nuvem do Google, que oferece sincronização em tempo real, autenticação integrada e escalabilidade gerenciada. A escolha por um banco em nuvem elimina a necessidade de infraestrutura de servidor própria durante a fase de desenvolvimento e prototipação.

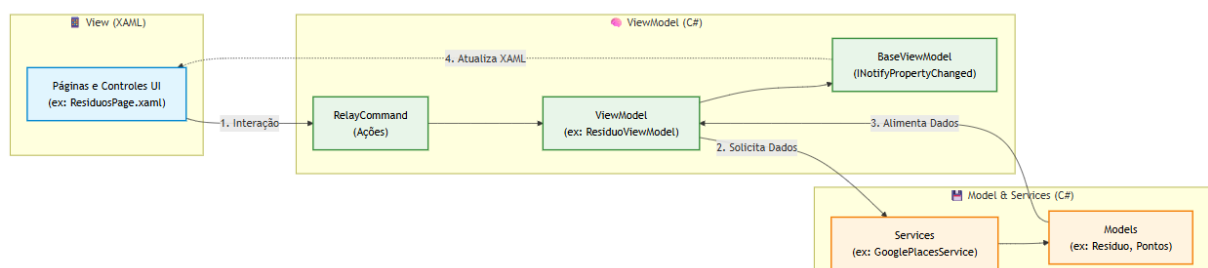
6.5 GOOGLE MAPS PLACES API

A Google Maps Places API fornece dados georreferenciados de estabelecimentos e pontos de coleta, permitindo ao ReGraphik localizar automaticamente os pontos mais próximos ao usuário. A renderização do mapa no cliente WPF é feita via Leaflet.js, carregado em um componente WebBrowser/WebView2.

7 ARQUITETURA DO PROJETO(MVVM)

Este projeto foi desenvolvido utilizando o padrão Model-View-ViewModel (MVVM), garantindo a separação clara entre a interface do usuário, a lógica de apresentação e as regras de negócio/dados.

Abaixo está o fluxo de comunicação entre as camadas da nossa aplicação:



8 MÓDULOS DO SISTEMA

O ReGraphik é organizado em módulos funcionais, cada um com escopo e status de desenvolvimento definidos. A tabela abaixo apresenta o panorama atual:

Módulo	Descrição	Status
Mapa / Pontos de Coleta	Integração com Google Places API para localização de pontos de coleta próximos ao usuário	Disponível
Dashboard	KPIs e indicadores de estoque reverso em tempo real	Em Construção
Cadastro de Resíduo	Triagem, classificação e registro de materiais com suporte a anexos	Em Construção
Sugestões	Motor de recomendação de reaproveitamento por tipo de material	Em Construção
Relatórios	Exportação de histórico e dados operacionais do sistema	Em Construção
Chat Interno	Canal de comunicação entre usuários da plataforma	Implementação Futura
Central de Notificações	Alertas automáticos de estoque e pendências operacionais	Implementação Futura
Log de Consulta	Auditoria e rastreabilidade completa de operações no sistema	Implementação Futura

8.1 MÓDULO: MAPA / PONTOS DE COLETA (DISPONÍVEL)

Este é o módulo mais maduro do sistema. Permite ao usuário buscar pontos de coleta seletiva próximos a uma cidade informada, utilizando a Google Maps Places API. Os resultados são exibidos em um mapa interativo renderizado via Leaflet.js. Os pontos de coleta são salvos automaticamente no Firebase para consultas futuras.

8.2 MÓDULO: DASHBOARD (EM CONSTRUÇÃO)

O Dashboard exibirá KPIs (Key Performance Indicators) e indicadores em tempo real do estoque reverso: total de resíduos cadastrados, quantidade reaproveitada, materiais pendentes e tendências mensais. A integração com o Firebase permitirá atualização automática dos indicadores.

8.3 MÓDULO: CADASTRO DE RESÍDUO (EM CONSTRUÇÃO)

O módulo de cadastro permite registrar cada resíduo gerado, informando tipo de material, especificação técnica, quantidade, condição, dimensões, observações e anexo fotográfico. O status inicial é automaticamente definido como 'Em Estoque'.

8.4 MÓDULO: SUGESTÕES (EM CONSTRUÇÃO)

O motor de sugestões relaciona automaticamente cada resíduo cadastrado às sugestões de reaproveitamento disponíveis para seu tipo de material. O usuário pode aplicar uma sugestão, gerando um registro em SugestoesResiduos com a data de aplicação.

8.5 MÓDULO: RELATÓRIOS (EM CONSTRUÇÃO)

O módulo de relatórios permitirá exportar o histórico de resíduos, sugestões aplicadas e indicadores de desempenho em formatos CSV ou PDF, possibilitando análise gerencial e prestação de contas ambiental.

9 PROCESSO DE DESENVOLVIMENTO

O desenvolvimento do ReGraphik foi organizado em três fases iterativas, cada uma com escopo definido e entregáveis concretos. O progresso é evidenciado pelo histórico de commits no repositório GitHub.

Fase	Commits	Atividades	Status
Fase 1 — Setup	30	Estrutura da solução C#, BaseViewModel, INotifyPropertyChanged, RelayCommand, modelagem inicial do banco	Concluído
Fase 2 — Core UI	80	WPF XAML (layout principal), tela de Login + ViewModel, Mapa de coleta + GooglePlacesService, início do CRUD de Resíduos	Concluído
Fase 3 — API REST	73+	ApiRestReGraphik criado, Controllers (Resíduos, Usuários, Sugestões), integração API ↔ WPF Client	Em Progresso

9.1 CONTROLE DE VERSÃO

O projeto utiliza Git para controle de versão, com repositório hospedado no GitHub (<https://github.com/BrunoMaiaSenai/ReGraphikApp>). Os 214 commits registrados até esta entrega demonstram o histórico consistente de desenvolvimento colaborativo da equipe.

9.2 AMBIENTE DE DESENVOLVIMENTO

- IDE: Visual Studio 2022 Community Edition;
- SDK: .NET 8 / .NET 9;
- Gerenciamento de pacotes: NuGet (backend) e npm (ferramentas auxiliares);
- Sistema operacional de desenvolvimento: Windows 11;
- Banco de dados de desenvolvimento: Firebase Realtime Database (nuvem).

10 BUSINESS MODEL CANVAS

Parceiros Chave	Atividades Chave	Proposta de Valor	Rel. com Clientes	Segmento de Clientes
· Gráficas e editoras · Cooperativas de reciclagem · Fornecedores de tecnologia	· Desenvolvimento do software · Pesquisa de reaproveitamento · Apoio técnico	· Gestão de estoque reverso · Redução de descarte e custos · Economia circular	· Atendimento personalizado · Suporte técnico contínuo · Treinamentos	· Gráficas pequenas/médias · Empresas comunicação visual · ESG compliance
	Recursos Chave		Canais	
	· Equipe técnica · Banco de dados · Algoritmos de sugestão		· Plataforma digital · Consultores comerciais · Redes sociais	
Estrutura de Custos			Fluxo de Receitas	
· Desenvolvimento e hospedagem do sistema · Equipe de tecnologia e suporte · Marketing e divulgação			· Licenciamento SaaS (mensal/anual) · Consultoria Técnica · Suporte Premium e personalizações	

11 VIABILIDADE ECONÔMICA

A análise de viabilidade econômica examina os custos envolvidos no desenvolvimento e operação do ReGraphik, os benefícios financeiros esperados com sua adoção e o retorno sobre o investimento para as empresas do setor gráfico que o utilizarem.

11.1 CUSTOS DE DESENVOLVIMENTO

Por tratar-se de um projeto acadêmico, os custos de desenvolvimento foram essencialmente de tempo e capacitação da equipe. A tabela abaixo projeta esses custos em valores de mercado, considerando a remuneração média de desenvolvedores júnior/pleno no Brasil em 2025:

Item	Horas Est.	Valor/Hora	Custo Total
Levantamento de requisitos e Mini Mundo	20 h	R\$ 60,00	R\$ 1.200,00
Modelagem de dados (Conceitual, Lógico, Físico)	30 h	R\$ 60,00	R\$ 1.800,00
Desenvolvimento da API REST (ASP.NET Core)	80 h	R\$ 80,00	R\$ 6.400,00

Desenvolvimento do cliente WPF (MVVM)	100 h	R\$ 80,00	R\$ 8.000,00
Integração Firebase + Google Maps API	40 h	R\$ 80,00	R\$ 3.200,00
Testes, correções e refatoração	30 h	R\$ 60,00	R\$ 1.800,00
Documentação técnica e TCC	30 h	R\$ 50,00	R\$ 1.500,00
TOTAL	330 h	—	R\$ 23.900,00

Por ser um projeto de TCC com equipe de cinco estudantes, o custo real de desenvolvimento foi zero em termos monetários diretos, representando apenas o investimento de tempo e aprendizado da equipe.

11.2 CUSTOS OPERACIONAIS

Os custos mensais de operação do sistema em produção são estimados abaixo, considerando uso por uma empresa de médio porte do setor gráfico com até 50 usuários:

Serviço	Custo Mensal Estimado	Observação
Firebase Realtime Database (plano Spark)	R\$ 0,00	Gratuito até 1 GB de armazenamento e 10 GB/mês de transferência
Firebase Realtime Database (plano Blaze)	R\$ 25,00 – R\$ 150,00	Cobrado por uso acima do plano gratuito; pay-as-you-go
Google Maps Places API	R\$ 0,00 – R\$ 100,00	Crédito de US\$ 200/mês cobre até ~13.000 buscas/mês
Hospedagem da API (Render / Railway / Azure Free)	R\$ 0,00 – R\$ 80,00	Planos gratuitos disponíveis para volume baixo e médio
Domínio e SSL (opcional)	R\$ 5,00 – R\$ 40,00	Let's Encrypt fornece SSL gratuito

TOTAL ESTIMADO	R\$ 0,00 – R\$ 370,00/mês	Cenário 100% gratuito viável para empresa de pequeno porte
-----------------------	----------------------------------	---

11.3 BENEFÍCIOS E RETORNO ESPERADO

A adoção do ReGraphik por empresas do setor gráfico pode gerar benefícios econômicos quantificáveis em múltiplas frentes. Os valores abaixo são estimativas baseadas em médias do setor gráfico brasileiro e nos dados da empresa AML.

11.3.1 REDUÇÃO DE CUSTOS COM DESCARTE

Empresas gráficas de médio porte gastam, em média, R\$ 800,00 a R\$ 3.000,00 mensais com coleta e descarte de resíduos sólidos. Com a gestão estruturada do ReGraphik, estima-se redução de 30% a 60% nesse custo pelo direcionamento correto dos materiais recicláveis.

Indicador	Sem ReGraphik	Com ReGraphik (estimativa)
Custo mensal de descarte (médio porte)	R\$ 1.500,00/mês	R\$ 600,00 – R\$ 900,00/mês
Economia mensal estimada	—	R\$ 600,00 – R\$ 900,00/mês
Economia anual estimada	—	R\$ 7.200,00 – R\$ 10.800,00/ano

11.3.2 RECEITA COM VENDA DE MATERIAIS RECICLÁVEIS

Materiais como aparas de papel, cartão e vinil possuem valor de mercado junto a cooperativas de reciclagem. Com o controle preciso proporcionado pelo ReGraphik, a empresa pode comercializar esses materiais de forma estruturada.

- Papel/cartão: R\$ 0,20 a R\$ 0,60/kg — empresa de médio porte pode gerar 500 a 2.000 kg/mês;

- Vinil/PVC: R\$ 0,30 a R\$ 0,80/kg — materiais com maior valor de revenda;
- Receita adicional estimada: R\$ 300,00 a R\$ 1.600,00 por mês, dependendo do volume e tipo de resíduo.

11.3.3 REAPROVEITAMENTO INTERNO — ECONOMIA CIRCULAR

O módulo de sugestões possibilita que a própria empresa utilize resíduos como matéria-prima para novos produtos (camisetas, canecas, brindes personalizados), eliminando o custo de aquisição de novos materiais e criando nova fonte de receita.

- Redução de até 20% no custo de matéria-prima para produtos secundários;
- Criação de linha de produtos sustentáveis com maior margem de lucro e diferencial competitivo;
- Melhoria na imagem da marca junto a clientes e parceiros — alinhamento com ESG.

11.4 ANÁLISE CUSTO-BENEFÍCIO

Categoria / Item	Tipo	Valor Anual Estimado
Desenvolvimento (amortizado em 3 anos)	Custo	R\$ 7.967,00/ano
Operação / Infraestrutura (cenário básico)	Custo	R\$ 0,00 – R\$ 1.200,00/ano
Manutenção e suporte técnico (estimado)	Custo	R\$ 1.200,00 – R\$ 2.400,00/ano
Redução de custos com descarte	Benefício	R\$ 7.200,00 – R\$ 10.800,00/ano
Receita com venda de recicláveis	Benefício	R\$ 3.600,00 – R\$ 19.200,00/ano
Economia em matéria-prima (reaproveitamento)	Benefício	R\$ 2.400,00 – R\$ 6.000,00/ano
SALDO LÍQUIDO ESTIMADO	Lucro	R\$ 4.033,00 – R\$ 24.433,00/ano

A análise demonstra que o investimento no desenvolvimento do ReGraphik tem retorno positivo já no primeiro ano de operação, com payback estimado entre 3 e 8 meses, dependendo do volume de resíduos gerados pela empresa.

Além dos benefícios econômicos diretos, a adoção do sistema contribui para o cumprimento da Política Nacional de Resíduos Sólidos (Lei nº 12.305/2010), que estabelece responsabilidades ambientais para as empresas e pode resultar em benefícios fiscais e certificações de sustentabilidade que agregam valor à marca.

12 ARTEFATOS TÉCNICOS ENTREGUES

Esta seção consolida os artefatos técnicos produzidos e entregues como parte do TCC, conforme os requisitos definidos pelo curso.

Artefato	Descrição	Status
Mini Mundo da Demanda	Contextualização do problema, atores e regras de negócio do sistema	Entregue
Modelo Conceitual (DER)	Diagrama ER com as 5 entidades do domínio e seus relacionamentos (BRModelo)	Entregue
Modelo Lógico	Esquema relacional normalizado com PK, FK e restrições de integridade	Entregue
Modelo Físico	Script SQL completo com tabelas, índices e triggers	Entregue
Swagger / OpenAPI	Documentação interativa de todos os endpoints REST da API	Entregue
Repositório GitHub	Código-fonte completo com histórico de 182+ commits	Entregue

Artefato	Descrição	Status
Apresentação MVVM	Slides técnicos detalhando arquitetura, padrões e integração	Entregue
Documentação TCC (este documento)	Documentação completa do sistema para entrega do TCC	Entregue
Diagrama MVVM	Diagrama das ViewModels, bindings e Commands	Entregue

13 REQUISITOS

13.1 REQUISITOS FUNCIONAIS

Os requisitos funcionais descrevem as ações, facilidades e comportamentos que o sistema ReGraphik deve oferecer aos seus usuários para atender às demandas de gestão de estoque reverso.

RF-001 – Autenticação e Controle de Acesso: O sistema deve permitir que usuários realizem login informando usuário (login) e senha criptografada, diferenciando as permissões entre perfis comuns e Administradores.

RF-002 – Cadastro e Triagem de Resíduos: O usuário deve ser capaz de registrar resíduos gráficos detalhando especificações técnicas, origem (setor/projeto), quantidade, dimensões (cm e metros lineares), estado físico e inclusão de anexos fotográficos.

RF-003 – Atualização Automática de Status: O sistema deve gerenciar e alterar o estado físico/comercial do resíduo entre os status predefinidos: *Em Estoque*, *Reaproveitado* e *Descartado*.

RF-004 – Motor de Sugestões de Reaproveitamento: O sistema deve listar e correlacionar automaticamente as sugestões de economia circular salvas com base na classificação do tipo de material selecionado.

RF-005 – Geolocalização de Pontos de Coleta: O sistema deve possibilitar a busca de pontos de reciclagem externos por meio da inserção do nome da cidade, consumindo dados georreferenciados em tempo real.

RF-006 – Sincronização Automática com Firebase: Toda consulta realizada via API de mapas que retorne novos pontos de coleta deve ser salva automaticamente no banco de dados para consultas offline subsequentes.

RF-007 – Auditoria e Exclusão Segura: A remoção física ou lógica de qualquer registro de resíduo deve ser restrita a administradores e gerar uma trilha de auditoria/log automática no sistema.

RF-008 – Geração de Relatórios e Dashboards: O sistema deve consolidar os dados operacionais em indicadores visuais gráficos (KPIs) na tela inicial e permitir a exportação dos dados tabulados em formatos abertos (como CSV).

13.2 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais determinam as restrições, qualidades e propriedades técnicas que o software deve possuir para operar com segurança e eficiência.

RNF-001 – Desempenho e Concorrência: A API REST deve utilizar o padrão de programação assíncrona (async/await) em C# para garantir que requisições ao banco de dados NoSQL não bloqueiem a thread principal do cliente desktop.

RNF-002 – Compatibilidade de Ambiente: O aplicativo cliente frontend deve funcionar obrigatoriamente sobre ambientes Microsoft Windows (Windows 10 ou 11 de 64 bits), utilizando os redistribuíveis nativos do .NET 8 Runtime.

RNF-003 – Disponibilidade do Banco de Dados: O armazenamento de dados em nuvem, delegado ao Firebase Realtime Database, deve seguir as diretrizes do plano de serviço que garante uma disponibilidade mínima (SLA).

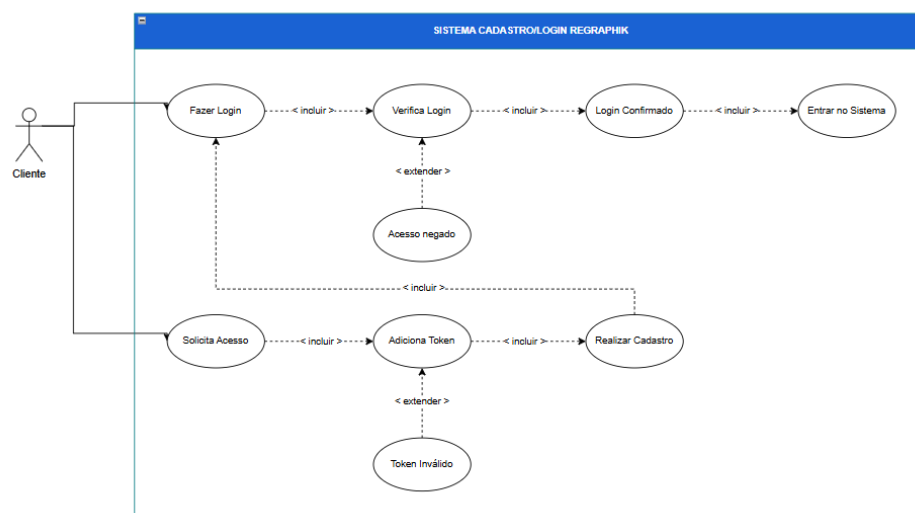
RNF-004 – Escalabilidade Arquitetural: A interface do cliente WPF deve ser implementada desacoplada da lógica através do padrão MVVM, permitindo substituições de serviços de infraestrutura (como a troca do provedor de mapas) via inversão/injeção de dependências sem afetar as Views.

RNF-005 – Renderização de Mapas Leve: A exibição cartográfica na interface desktop deve ser realizada por meio de injeção de scripts JavaScript usando a biblioteca Leaflet.js rodando nativamente dentro do componente WebView2/WebBrowser.

RNF-006 – Segurança da Informação: As senhas dos usuários não podem ser guardadas em texto claro, sendo obrigatório o uso de algoritmos de dispersão (hash) criptografados antes da persistência.

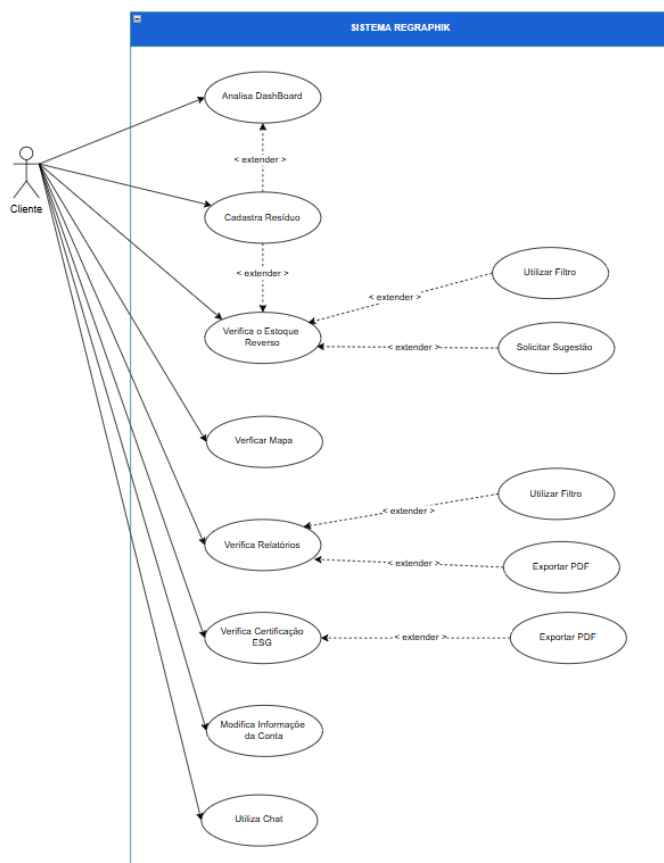
14 CASO DE USO

14.1 Diagrama de Caso de uso do Login



14.2

Diagrama de Caso de uso do Sistema Completo



15 HISTORICO DE EVOLUÇÃO E ATUALIZAÇÃO DO PROJETO

15.1 FASE DE INICIALIZAÇÃO E INFRAESTRUTURA BASE(SETUP INICIAL)

Estruturação da Solução Multicamadas: Criação da arquitetura desacoplada no Visual Studio dividindo o projeto em duas frentes de execução: o backend de microsserviços (ApiRestReGraphik) e o cliente rico desktop (ReGraphik).

Implementação da Arquitetura MVVM Nativa: Desenvolvimento e consolidação da infraestrutura necessária para o funcionamento do padrão MVVM no WPF, com a criação da classe abstrata BaseViewModel encapsulando a interface de notificação de propriedades (INotifyPropertyChanged), e a classe genérica RelayCommand para substituição completa de manipuladores de eventos em code-behind por binds limpos em XAML.

15.2 FASE DE AMADURECIMENTO DO FRONTEND E INTEGRAÇÃO GEOGRÁFICA

Concepção do Design e Views Operacionais: Codificação das interfaces com bindings bi-direcionais declarativos para as telas críticas, incluindo a página de autenticação segura, o painel gerencial de resíduos e o módulo cartográfico interativo.

Acoplamento do Leaflet.js com WebView2: Substituição de controles estáticos por uma rotina dinâmica de injeção de scripts capaz de gerar arquivos HTML temporários. Essa atualização viabilizou a renderização fluida e responsiva de mapas interativos dentro do ambiente Windows Desktop, integrando marcadores de latitude e longitude sem custos adicionais de licenciamento.

Consumo de Microsserviços de Geolocalização: Desenvolvimento do serviço local GooglePlacesService estruturado sob a abstração de interfaces (IPlacesService), isolando as chaves de API externas e protegendo a lógica interna contra futuras quebras de contrato de fornecedores.

15.3 FASE DE CONSTRUÇÃO DO BACKEND, PERSISTENCIA E SINCRONIZAÇÃO EM NUVEM

Migração e Integração do Firebase SDK: Evolução do armazenamento local para persistência global em nuvem. Configuração da classe de contexto DbReGraphik.cs conectando a API REST diretamente às chaves de serviço em formato JSON do Google Service Account, permitindo transações seguras sob protocolo OAuth 2.0.

Desenvolvimento e Modularização dos Controllers: Codificação progressiva de 5 controladores REST (UsuarioController, PontosColetaController, ResiduosController, SugestaoController e SugestaoResiduosController) responsáveis por processar as chamadas assíncronas do aplicativo cliente e aplicar as devidas regras de validação lógica de dados.

Implementação de Mecanismo Anti-Redundância Inteligente: Atualização no endpoint /api/PontosColeta/google para injetar uma verificação prévia no Firebase Realtime Database antes de disparar requisições pagas à API de mapas, otimizando os custos operacionais e respeitando os limites da cota gratuita mensal estipulada para o projeto.

Padronização e Entrega da Documentação Swagger UI: Acoplamento e configuração do pacote Swashbuckle na inicialização da API (Program.cs), gerando uma interface interativa completa exposta na rota raiz /swagger, validando os esquemas JSON de payload e permitindo auditoria externa imediata de todos os endpoints CRUD disponíveis no sistema.

16 REGRA DE NEGÓCIO

Autenticação e cadastro

- Solicitação de acesso/pre-cadastro (cliente → ApiConfig):
 - Usuário preenche formulário com Nome, CPF, Email, Login, Senha.
 - ApiConfig/UsuarioCadastro gera token numérico de 6 dígitos (TokenService => Random.Next(100000,999999)) e cria registro de pré-cadastro com Ativo = false e TokenValidacao armazenado.
 - ApiConfig envia token por e-mail ao endereço indicado.
- Validação de token (cliente → ApiConfig/AcessoCadastroController):
 - Endpoint /api/AcessoCadastro/validar-token recebe Email + Token.
 - Se token bate com TokenValidacao do usuário, marca usuario.Ativo = true.
- Finalização do cadastro (cliente → ApiRestReGraphik/api/Usuario/finalizar-cadastro?token=...):

- ApiRestReGraphik mantém um dicionário (`_tokensAutorizados`) de tokens esperados por email.
- Requisição POST com `UsuarioDto` e token (query param) — a API só cria o usuário se o token na memória corresponder ao token recebido para esse email.
- Ao criar usuário, define Id (GUID), `DataCadastro = UTC now`, `Ativo = true` e Perfil conforme dto (padrão "User").
- Login:
 - POST `/api/Usuario/login` com login+senha. Se sucesso, retorna objeto `Usuario` (dados do usuário).
- Regras de perfil:
 - Perfil é string; valores observados: "User" (default), menções a "Admin"/"Administrador"/"Guest" nos comentários. Implementação de autorização fina não foi evidente no trecho analisado — é possível que o controle de permissões seja tratado em camada superior ou ainda não implementado.

Validação de dados

- CPF:
 - Validacao local: `ValidacaoCpfService.Validar(cpf)` implementa algoritmo oficial de dígitos verificadores (remover não-numéricos, validar tamanho 11, checar dígitos verificadores).
 - Formatação por `Formatar(cpf) => 000.000.000-00`.
- Upload de arquivos:
 - Foto de perfil: seletor de arquivos cliente filtra extensões: *.jpg; *.jpeg; *.png; *.bmp.
 - Em código houve alteração para "aceitar apenas imagens" — regra: bloquear uploads de outros tipos.
- Validação de token de cadastro:
 - Token de 6 dígitos, gerado com Random, enviado por e-mail e checado por duas fases (ApiConfig grava token no usuário de pré-cadastro;

ApiRestReGraphik mantém tokens autorizados em memória para finalizar cadastro).

Resíduos (cadastro / consulta / fluxo)

- CRUD de resíduos exposto via API (ApiRestReGraphik/api/Residuo).
- Propriedades do resíduo (observáveis no ViewModel): TipoMaterial, Especificacao, Origem, ProjetoOrigem, Quantidade, Data, Condicao, Comprimento, Largura, Observacoes, etc.
- Regras de cadastro no cliente (ResiduoViewModel):
 - Campos obrigatórios validados antes do POST.
 - Upload de arquivos (anexos/imagens) com validação de tipo e salvamento.
- Estoque reverso:
 - Visualização do estoque como cards (UI).
 - Estado e cores indicam status — regras de mapeamento status->cor definidas no client (ex.: corrigir cores de status).
- Relatório/estatísticas:
 - Relatórios filtram por Tipo, Status, Origem, Data inicio/fim.
 - Calculam: TotalResiduos, PesoTotal, Reaproveitados, ValorEconomico (formato monetário), TotalRegistros.
 - Exportação: permitir imprimir ou salvar PDF via diálogo de impressão (PrintDialog) ou via QuestPDF.

Sugestões (Diretório de ideias / aplicação)

- Sugestao (modelo): TipoResiduoAceito, DescricaoSugestao, Id.
- SugestaoResiduos (registro de aplicação): IdCadastroResiduo, IdSugestao, DataAplicacao; servidores associam objetos Sugestao e CadastroResiduo no retorno (enriquecimento).
- Fluxo:
 - API /api/Sugestao: CRUD de sugestões (lista, get por id, criar, atualizar, deletar).

- API /api/SugestaoResiduos: registra quando uma sugestão foi aplicada a um resíduo (POST), lista histórico de aplicações.
- Regras de filtragem:
 - No cliente (SugestaoResiduoViewModel) ao selecionar um resíduo o sistema filtra as sugestões por material — commit observa que a filtragem foi otimizada para aceitar correspondências parciais e bidirecionais (ex.: "Papel" ↔ "Papel Couché").
- Estado atual do produto:
 - UI recente converteu a janela de sugestões para modo "consulta" (removido o campo de texto manual e o botão de aplicar) — regra de produto: somente leitura/diretório de ideias (possível decisão de negócios por segurança/fluxo).
- Evento de compatibilidade:
 - Event SugestaoAplicadaComSucesso foi mantido para compatibilidade com código mais antigo.

Chat (mensageria)

- Armazenamento: Firebase Realtime Database.
- Convo ID determinístico:
 - ConversalId(id1,id2) => ordenar alfabeticamente e concatenar: \${menor}_\${maior}. Regra: mesma conversa independente de quem iniciou.
- Envio de mensagem:
 - EnviarMensagemAsync persiste a mensagem em NodeMensagens/{convId}/{mensagem.Id}.
- ObterMensagens:
 - Lista mensagens do node convId ordenadas por DataHora.
- Marcar como lida:
 - MarcarComoLidaAsync(remetentId,destinatariId) — busca mensagens no convId e marca como lida (provisoriamente o método filtra mensagens cujo RemetentId == remetentId e !Lida).

- Regra operacional: ao abrir/ler conversa, marcar as mensagens pendentes do outro usuário como lidas.
- Listar usuários:
 - Faz leitura do node "usuarios", garante que o Id venha da chave do Firebase se campo interno estiver vazio.

Pontos de coleta (Google Places)

- Endpoint API: POST /api/PontosColeta/google?cidade=...
 - Valida se cidade já está cadastrada antes de consultar Google Places.
 - Se autorizado, busca pontos via Google Places API, salva no Firebase e retorna lista com IDs gerados.
- Regra: evitar duplicidade de cidades + cache de pontos antes de requisição externa.

Configuração local / foto de perfil

- ConfiguracaoLocalService grava caminho do arquivo da foto em %AppData%/ReGraphik/config.txt.
- Ao iniciar, ViewModel carrega o caminho salvo e exibe foto (se existir). Regra: persistir caminho entre sessões; se o arquivo não existir, retornar null.

Exportação / impressão

- Relatórios e ESG exportáveis via PrintDialog (usuário escolhe impressora ou salvar como PDF). Regra: usar diálogo nativo para compatibilidade com gerar PDF localmente.

Regras de segurança observadas / faltantes

- Observadas:
 - Token de validação por e-mail para ativar contas (6 dígitos).
 - Validação local de CPF.
 - Filtragem de tipos de arquivo nas seleções do cliente.

Regras operacionais (resumo objetivo - prontas para documentação)

- Cadastro:
 - Pré-cadastro cria TokenValidacao (6 dígitos) e Ativo=false → enviar token por e-mail.
 - Validação do token → Ativo=true no pré-cadastro (ApiConfig) e autorização em memória para finalizar cadastro (ApiRestReGraphik).
- Login:
 - Autenticar via POST /api/Usuario/login (login + senha).
- Usuário:
 - Perfil default: "User"; possíveis valores: "Admin", "Guest". DataCadastro gerada automaticamente.
- Resíduo:
 - CRUD via API; ao cadastrar validar campos obrigatórios; imagens permitidas somente em tipos pré-definidos.
- Sugestão:
 - Diretório de sugestões: cadastrado via API; filtrado localmente por tipo de resíduo; sugestão aplicada pode ser registrada como SugestaoResiduo (com DataAplicacao).
- Chat:
 - Mensagens salvas por convld determinístico; marcação de lida atualiza campo "lida" no Firebase.
- Ponto de Coleta:
 - Buscar/Salvar via Google Places sendo aplicada checagem de duplicidade por cidade.
- Relatórios/ESG:
 - Exportação via diálogo de impressão; relatórios calculam totais e valor econômico formatado em R\$.

16 CONSIDERAÇÕES FINAIS

O desenvolvimento do ReGraphik ao longo do curso técnico em Desenvolvimento de Sistemas do SENAI proporcionou à equipe uma experiência abrangente de engenharia de software aplicada a um problema real. O projeto combinou conceitos de modelagem de dados, arquitetura de software, desenvolvimento de APIs REST, programação desktop e integração com serviços externos em nuvem.

Os principais aprendizados obtidos durante o desenvolvimento incluem: a aplicação prática do padrão arquitetural MVVM em uma aplicação WPF real; o design e implementação de uma API REST com ASP.NET Core, incluindo injeção de dependências e programação assíncrona; a modelagem relacional em três níveis (conceitual, lógico e físico); e a integração com serviços de nuvem como Firebase e Google Maps Places API.

O sistema cumpre os objetivos propostos ao oferecer uma solução concreta para o problema de gestão inadequada de resíduos no setor gráfico. O módulo de mapa, já funcional, demonstra a viabilidade técnica da integração entre o cliente desktop, a API REST e os serviços externos. Os demais módulos, atualmente em construção, têm sua arquitetura estabelecida e prontos para implementação.

Como trabalhos futuros, destacam-se: a implementação completa dos módulos de Dashboard, Cadastro de Resíduo, Sugestões e Relatórios; a adição de autenticação JWT na API REST; o desenvolvimento do diagrama MVVM formal; a ampliação dos testes unitários com mocks para os serviços; e a eventual publicação da aplicação em ambiente de produção para uso real pela empresa AML.

O ReGraphik representa não apenas um exercício técnico, mas uma proposta de contribuição real à sustentabilidade no setor gráfico, demonstrando como o desenvolvimento de sistemas pode gerar valor econômico e ambiental concreto para as organizações.

REFERÊNCIAS

MICROSOFT. ASP.NET Core documentation. Disponível em: <https://learn.microsoft.com/aspnet/core>. Acesso em: 2025.

MICROSOFT. Windows Presentation Foundation. Disponível em: <https://learn.microsoft.com/dotnet/desktop/wpf>. Acesso em: 2025.

GOOGLE. Firebase Realtime Database. Disponível em: <https://firebase.google.com/docs/database>. Acesso em: 2025.

GOOGLE. Places API (New). Disponível em: <https://developers.google.com/maps/documentation/places/web-service>. Acesso em: 2025.

AGAMBARD, T. et al. Leaflet.js — an open-source JavaScript library for mobile-friendly interactive maps. Disponível em: <https://leafletjs.com>. Acesso em: 2025.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

MICROSOFT. Model-View-ViewModel (MVVM). Disponível em: <https://learn.microsoft.com/dotnet/architecture/maui/mvvm>. Acesso em: 2025.

RICHARDSON, Leonard. RESTful Web Services. Sebastopol: O'Reilly Media, 2007.

SWAGGER. OpenAPI Specification. Disponível em: <https://swagger.io/specification>. Acesso em: 2025.

BRMODELO. Ferramenta de Modelagem Entidade-Relacionamento. Disponível em: <https://www.sis4.com/brModelo>. Acesso em: 2025.

